

Simply-typed Lambda Calculus

Stefan Muller

CS 5095: Types and Programming Languages, Fall 2026
Lectures 9-10

1 Function and Unit Types

We're going to start adding types to the (formerly untyped) lambda calculus. The first type we'll add is a type of functions $\tau_1 \rightarrow \tau_2$, which is a function from arguments of type τ_1 to results of type τ_2 . The arrow will associate to the right, so the type $\tau_1 \rightarrow \tau_2 \rightarrow \tau_3$ means a function that takes a τ_1 and returns a function that takes a τ_2 and returns a τ_3 , which is how we implemented multi-argument functions in lambda calculus. The types τ_1 and τ_2 can themselves be functions, but this recursion needs to bottom out somewhere, so we need another type. We'll add a "base type" **unit** (sometimes notated 1) with one value $()$ (also pronounced "unit"). In C, you might think of this as the type "void." We'll see soon why we **don't** call this "void." Expressions e are the same as lambda terms but now we actually say what the type of a function argument is (like we do in C, Java, etc.)

We'll keep adding to this, but this is the language for now:

Expressions $e ::= x \mid () \mid \lambda x : \tau. e \mid e \ e$
Types $\tau ::= \text{unit} \mid \tau \rightarrow \tau$

And here are the static and dynamic rules for now:

$$\begin{array}{c}
 \frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau} \text{ (VAR)} \qquad \frac{}{\Gamma \vdash () : \text{unit}} \text{ (unit-I)} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1 \ e_2 : \tau_2} (\rightarrow\text{-E}) \\
 \\
 \frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x : \tau. e : \tau \rightarrow \tau'} (\rightarrow\text{-I}) \\
 \\
 \frac{}{() \text{ val}} \text{ (VALUNIT)} \qquad \frac{}{\lambda x : \tau. e \text{ val}} \text{ (VALLAMBDA)} \qquad \frac{e_1 \mapsto e'_1}{e_1 \ e_2 \mapsto e'_1 \ e_2} \text{ (STEPSEARCHAPPLEFT)} \\
 \\
 \frac{e_2 \mapsto e'_2}{(\lambda x : \tau. e) \ e_2 \mapsto (\lambda x : \tau. e) \ e'_2} \text{ (STEPSEARCHAPPRIGHT)} \qquad \frac{v \text{ val}}{(\lambda x : \tau. e) \ v \mapsto [v/x]e} \text{ (STEPAPP)}
 \end{array}$$

Note that for the statics, instead of using more descriptive names, we're now labeling the rules with the type they deal with ($\rightarrow$ or **unit**) and whether they Introduce or Eliminate expressions of that type.

Example.

$(\lambda x : \text{unit}. x) \ ()$

$$\frac{\frac{\frac{}{x : \text{unit} \vdash x : \text{unit}} \text{ (VAR)}}{\bullet \vdash \lambda x : \text{unit}. x : \text{unit} \rightarrow \text{unit}} (\rightarrow\text{-I}) \quad \frac{}{\bullet \vdash () : \text{unit}} \text{ (unit-I)}}{\bullet \vdash (\lambda x : \text{unit}. x) \ () : \text{unit}} (\rightarrow\text{-E})$$

$$\begin{array}{c} (\lambda x : \text{unit}.x) () \\ \mapsto () \end{array}$$

Example.

$$(\lambda f : \text{unit} \rightarrow \text{unit}.f ()) (\lambda x : \text{unit}.x) : \text{unit}$$

$(\lambda f : \text{unit} \rightarrow \text{unit}.f ()) (\lambda x : \tau.x)$ for any $\tau \neq \text{unit}$ isn't well-typed: the type annotations matter!

2 Products

$$\begin{array}{ll} \text{Expressions } e & ::= \dots \mid (e, e) \mid \text{fst } e \mid \text{snd } e \\ \text{Types } \tau & ::= \dots \mid \tau \times \tau \end{array}$$

Examples (Let's assume we also have `int` and `string` as base types for these.)

$$\begin{array}{ll} (\bar{1}, \bar{2}) & : \text{int} \times \text{int} \\ (\bar{1}, \text{"Hi"}) & : \text{int} \times \text{string} \\ (\text{"Hi"}, \bar{1}) & : \text{string} \times \text{int} \\ ((), \bar{1}) & : \text{unit} \times \text{int} \\ \lambda x : \text{unit}.\lambda y : \text{int}.(x, y) & : \text{unit} \rightarrow \text{int} \rightarrow (\text{unit} \times \text{int}) \\ ((\lambda x : \text{unit}.x), \bar{1}) & : ((\text{unit} \rightarrow \text{unit}), \text{int}) \\ \lambda x : \text{int} \times \text{string}.\text{fst } x & : \text{int} \times \text{string} \rightarrow \text{int} \end{array}$$

$$\begin{array}{lll} \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1, e_2) : \tau_1 \times \tau_2} (\times\text{-I}) & \frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \text{fst } e : \tau_1} (\times\text{-E1}) & \frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \text{snd } e : \tau_2} (\times\text{-E2}) \\ \\ \frac{v_1 \text{ val} \quad v_2 \text{ val}}{(v_1, v_2) \text{ val}} (\text{VALPAIR}) & \frac{e_1 \mapsto e'_1}{(e_1, e_2) \mapsto (e'_1, e_2)} (\text{STEPSEARCHPAIRLEFT}) & \\ \\ \frac{v_1 \text{ val} \quad e_2 \mapsto e'_2}{(v_1, e_2) \mapsto (v_1, e'_2)} (\text{STEPSEARCHPAIRRIGHT}) & \frac{e \mapsto e'}{\text{fst } e \mapsto \text{fst } e'} (\text{STEPSEARCHFST}) & \\ \\ \frac{v_1 \text{ val} \quad v_2 \text{ val}}{\text{fst } (v_1, v_2) \mapsto v_1} (\text{STEPFST}) & \frac{e \mapsto e'}{\text{snd } e \mapsto \text{snd } e'} (\text{STEPSEARCHSND}) & \frac{v_1 \text{ val} \quad v_2 \text{ val}}{\text{snd } (v_1, v_2) \mapsto v_2} (\text{STEPSND}) \end{array}$$

3 Sums

$$\begin{array}{ll} \text{Expressions } e & ::= \dots \mid \text{inl } e \mid \text{inr } e \mid \text{case } e \text{ of } \{x.e; y.e\} \\ \text{Types } \tau & ::= \tau + \tau \end{array}$$

`int + string`: can be an `int` or a `string` (but you have to say which one)

Examples.

$$\begin{array}{ll} \text{inl } \bar{1} & : \text{int} + \text{string} \\ \text{inl } \bar{1} & : \text{int} + \text{int} \\ \text{inl } \bar{1} & : \text{int} + \text{unit} \\ \text{inr "Hi"} & : \text{int} + \text{string} \\ \text{case inl } \bar{1} \text{ of } \{x.x; y.|y|\} & : \text{int} \\ \text{case inr "Hi"} \text{ of } \{x.x; y.|y|\} & : \text{int} \\ \lambda x : \text{int} + \text{string}.\text{case } x \text{ of } \{x.x; y.|y|\} & : (\text{int} + \text{string}) \rightarrow \text{int} \end{array}$$

$$\begin{array}{c}
\frac{\Gamma \vdash e : \tau_1}{\Gamma \vdash \text{inl } e : \tau_1 + \tau_2} (+\text{I1}) \qquad \frac{\Gamma \vdash e : \tau_2}{\Gamma \vdash \text{inr } e : \tau_1 + \tau_2} (+\text{I2}) \\
\\
\frac{\Gamma \vdash e : \tau_1 + \tau_2 \quad \Gamma, x : \tau_1 \vdash e_1 : \tau \quad \Gamma, y : \tau_2 \vdash e_2 : \tau}{\Gamma \vdash \text{case } e \text{ of } \{x.e_1; y.e_2\} : \tau} (+\text{E}) \\
\\
\frac{v \text{ val}}{\text{inl } v \text{ val}} (\text{VALINL}) \qquad \frac{v \text{ val}}{\text{inr } v \text{ val}} (\text{VALINR}) \qquad \frac{e \mapsto e'}{\text{inl } e \mapsto \text{inl } e'} (\text{STEPSEARCHINL}) \\
\\
\frac{e \mapsto e'}{\text{inr } e \mapsto \text{inr } e'} (\text{STEPSEARCHINR}) \qquad \frac{e \mapsto e'}{\text{case } e \text{ of } \{x.e_1; y.e_2\} \mapsto \text{case } e' \text{ of } \{x.e_1; y.e_2\}} (\text{STEPSEARCHCASE}) \\
\\
\frac{v \text{ val}}{\text{case inl } v \text{ of } \{x.e_1; y.e_2\} \mapsto [v/x]e_1} (\text{STEPCASEL}) \qquad \frac{v \text{ val}}{\text{case inr } v \text{ of } \{x.e_1; y.e_2\} \mapsto [v/y]e_2} (\text{STEPCASER}) \\
\\
\begin{array}{l}
\text{case inr "Hi" of } \{x.x; y.|y|\} \\
\mapsto \lfloor \text{"Hi"} \rfloor \\
\mapsto \frac{1}{2}
\end{array}
\end{array}$$

Example: Options (like null pointer)

$$\begin{array}{l}
\text{int option} \triangleq \text{int} + () \\
\text{case } e \text{ of } \{x.x; y.\bar{0}\}
\end{array}$$

4 Type Safety

Lemma 1 (Canonical Forms).

1. If $e \text{ val}$ and $\bullet \vdash e : \text{unit}$, then $e = ()$.
2. If $e \text{ val}$ and $\bullet \vdash e : \tau_1 \rightarrow \tau_2$, then $e = \lambda x. \tau_1 e_0$ for some e_0 .
3. If $e \text{ val}$ and $\bullet \vdash e : \tau_1 \times \tau_2$, then $e = (v_1, v_2)$ where $v_1 \text{ val}$ and $v_2 \text{ val}$.
4. If $e \text{ val}$ and $\bullet \vdash e : \tau_1 + \tau_2$, then $e = \text{inl } v$ where $v \text{ val}$ or $e = \text{inr } v$ where $v \text{ val}$.

Theorem 1 (Progress). If $\bullet \vdash e : \tau$ then either $e \text{ val}$ or there exists e' such that $e \mapsto e'$.

Proof. By induction on the derivation of $\bullet \vdash e : \tau$. We'll just do one case as an example.

- $+\text{E}$. Then $e = \text{case } e_1 \text{ of } \{x.e_2; y.e_3\}$ and $\bullet \vdash e_1 : \tau_1 + \tau_2$ and $x : \tau_1 \vdash e_2 : \tau$ and $y : \tau_2 \vdash e_3 : \tau$. By induction, $e_1 \text{ val}$ or there exists $e_1 \mapsto e'_1$.
 - $e_1 \text{ val}$. Then, by canonical forms, $e_1 = \text{inl } v$ or $e_2 = \text{inr } v$, and e steps by (STEPCASEL) or (STEPCASER).
 - $e_1 \mapsto e'_1$. Then e steps by (STEPSEARCHCASE).

□

Theorem 2 (Preservation). If $\bullet \vdash e : \tau$ and $e \mapsto e'$ then $\bullet \vdash e' : \tau$.

Proof. By induction on the derivation of $e \mapsto e'$. We'll just do a few cases.

- **STEPSEARCHAPPLEFT**. Then $e = e_1 \ e_2$. By inversion, $\bullet \vdash e_1 : \tau' \rightarrow \tau$. By induction, $\bullet \vdash e'_1 : \tau' \rightarrow \tau$. Apply $\rightarrow$ -E.
- **STEPSEARCHAPPRIGHT**. Similar.
- **STEPAPP**. Then $e = (\lambda x : \tau'. e_0) \ v$ and $e' = [v/x]e_0$. By inversion on $\rightarrow$ -E, $\bullet \vdash \lambda x : \tau'. e_0 : \tau' \rightarrow \tau$ and $\bullet \vdash v : \tau'$. By inversion on $\rightarrow$ -I, $x : \tau' \vdash e_0 : \tau$. By substitution, $\bullet \vdash [v/x]e_0 : \tau$.
- **STEPFST**. Then $e = \text{fst} \ (e', v_2)$. By inversion on $\times$ -E1, $\bullet \vdash (e', v_2) : \tau \times \tau_2$. By inversion on $\times$ -I, $\bullet \vdash e' : \tau$.
- **STEPCASEL**. Then $e = \text{case inl } v \text{ of } \{x.e_1; y.e_2\}$ and $v \text{ val}$ and $e' = [v/x]e_1$. By inversion, $\bullet \vdash \text{inl } v : \tau_1 + \tau_2$ and $x : \tau_1 \vdash e_1 : \tau$. By another inversion, $\bullet \vdash v : \tau_1$. By substitution, $\bullet \vdash [v/x]e_1 : \tau$.

□

Fun fact If $\bullet \vdash e : \tau$ then there exists v such that $v \text{ val}$ and $e \mapsto^* v$. (We won't prove it now; the proof is a little tricky and uses proof techniques we haven't seen yet; we might see it a bit later in the class.)

Remember that this wasn't true for the untyped lambda calculus. So why is it true now that we've added types? Remember our self-application trick. Let's try to figure out the type of $(\lambda x.x \ x) \ (\lambda x.x \ x)$. There's no type annotation, so let's just say the type of x is τ and we'll figure it out later. We'll also say the return type of each function is τ' .

$$\begin{array}{c}
 \frac{}{x : \tau \vdash x : \tau \rightarrow \tau} \text{ (???)} \quad x : \tau \vdash x : \tau \\
 \hline
 x : \tau \vdash x \ x : \tau' \\
 \hline
 \bullet \vdash \lambda x.x \ x : \tau \rightarrow \tau' \quad \dots \\
 \hline
 \bullet \vdash (\lambda x.x \ x) \ (\lambda x.x \ x) : ???
 \end{array}$$

5 Observations

Booleans

<code>bool</code>	$\triangleq$	<code>unit + unit</code>
<code>true</code>	$\triangleq$	<code>inl ()</code>
<code>false</code>	$\triangleq$	<code>inr ()</code>
<code>if e_1 then e_2 else e_3 fi</code>	$\triangleq$	<code>case e_1 of $\{x.e_2; y.e_3\}$</code>

- The type of Booleans is sometimes called 2 because it has 2 elements.
- We write $\cong$ to mean that two types are “equal” (really isomorphic, but we're not going to go into the definition of that).
- So $2 \cong 1 + 1$!
- This isn't a coincidence. Think about how many elements, e.g., the type 2×2 has (i.e., how many different pairs of Booleans there are).

More general sums and products

- *Binary products* $\tau_1 \times \tau_2$:
 - 1 intro form (way to create)—pair
 - 2 elim forms (things to do with them)—projections `fst`, `snd`
- *n-ary products* $\tau_1 \times \dots \times \tau_n$:
 - 1 intro form (way to create)—“tuple”

- n elim forms (things to do with them)—projections π_i
- (We can also just encode this as nested binary products: $\tau_1 \times (\tau_2 \times (\cdots \times (\tau_{n-1} \times \tau_n)))$)
- *Binary sums* $\tau_1 + \tau_2$:
 - 2 intro forms (inl, inr)
 - 1 elim form (case)
- *n -ary sums* $\tau_1 + \cdots + \tau_n$:
 - n intro forms (“injections”)
 - 1 elim form (case)
- 0-ary (“nullary”) product?
 - 1 intro form (way to create)—“tuple”
 - 0 elim forms—nothing to do with it
 - Unit!
- Nullary sum?
 - 0 intro forms
 - 1 thing to do
 - void

6 Inference Rules (including void)

$$\begin{array}{c}
\frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau} \text{ (VAR)} \qquad \frac{}{\Gamma \vdash () : \text{unit}} \text{ (unit-I)} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1 \ e_2 : \tau_2} (\rightarrow\text{-E}) \\
\\
\frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x : \tau. e : \tau \rightarrow \tau'} (\rightarrow\text{-I}) \qquad \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1, e_2) : \tau_1 \times \tau_2} (\times\text{-I}) \qquad \frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \text{fst } e : \tau_1} (\times\text{-E1}) \\
\\
\frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \text{snd } e : \tau_2} (\times\text{-E2}) \qquad \frac{\Gamma \vdash e : \tau_1}{\Gamma \vdash \text{inl } e : \tau_1 + \tau_2} (+\text{-I1}) \qquad \frac{\Gamma \vdash e : \tau_2}{\Gamma \vdash \text{inr } e : \tau_1 + \tau_2} (+\text{-I2}) \\
\\
\frac{\Gamma \vdash e : \tau_1 + \tau_2 \quad \Gamma, x : \tau_1 \vdash e_1 : \tau \quad \Gamma, y : \tau_2 \vdash e_2 : \tau}{\Gamma \vdash \text{case } e \text{ of } \{x.e_1; y.e_2\} : \tau} (+\text{-E}) \qquad \frac{\Gamma \vdash e : \text{void}}{\Gamma \vdash \text{abort } e : \tau} (\text{void-E}) \\
\\
\frac{}{() \text{ val}} \text{ (VALUNIT)} \qquad \frac{}{\lambda x : \tau. e \text{ val}} \text{ (VALLAMBDA)} \qquad \frac{v_1 \text{ val} \quad v_2 \text{ val}}{(v_1, v_2) \text{ val}} \text{ (VALPAIR)} \\
\\
\frac{v \text{ val}}{\text{inl } v \text{ val}} \text{ (VALINL)} \qquad \frac{v \text{ val}}{\text{inr } v \text{ val}} \text{ (VALINR)} \\
\\
\frac{e_1 \mapsto e'_1}{e_1 \ e_2 \mapsto e'_1 \ e_2} \text{ (STEPSEARCHAPPLEFT)} \qquad \frac{e_2 \mapsto e'_2}{(\lambda x : \tau. e) \ e_2 \mapsto (\lambda x : \tau. e) \ e'_2} \text{ (STEPSEARCHAPPRIGHT)} \\
\\
\frac{v \text{ val}}{(\lambda x : \tau. e) \ v \mapsto [v/x]e} \text{ (STEPAPP)} \qquad \frac{e_1 \mapsto e'_1}{(e_1, e_2) \mapsto (e'_1, e_2)} \text{ (STEPSEARCHPAIRLEFT)} \\
\\
\frac{v_1 \text{ val} \quad e_2 \mapsto e'_2}{(v_1, e_2) \mapsto (v_1, e'_2)} \text{ (STEPSEARCHPAIRRIGHT)} \qquad \frac{e \mapsto e'}{\text{fst } e \mapsto \text{fst } e'} \text{ (STEPSEARCHFST)} \\
\\
\frac{v_1 \text{ val} \quad v_2 \text{ val}}{\text{fst } (v_1, v_2) \mapsto v_1} \text{ (STEPFST)} \qquad \frac{e \mapsto e'}{\text{snd } e \mapsto \text{snd } e'} \text{ (STEPSEARCHSND)} \qquad \frac{v_1 \text{ val} \quad v_2 \text{ val}}{\text{snd } (v_1, v_2) \mapsto v_2} \text{ (STEPSND)} \\
\\
\frac{e \mapsto e'}{\text{inl } e \mapsto \text{inl } e'} \text{ (STEPSEARCHINL)} \qquad \frac{e \mapsto e'}{\text{inr } e \mapsto \text{inr } e'} \text{ (STEPSEARCHINR)} \\
\\
\frac{e \mapsto e'}{\text{case } e \text{ of } \{x.e_1; y.e_2\} \mapsto \text{case } e' \text{ of } \{x.e_1; y.e_2\}} \text{ (STEPSEARCHCASE)} \\
\\
\frac{v \text{ val}}{\text{case inl } v \text{ of } \{x.e_1; y.e_2\} \mapsto [v/x]e_1} \text{ (STEPCASEL)} \qquad \frac{v \text{ val}}{\text{case inr } v \text{ of } \{x.e_1; y.e_2\} \mapsto [v/y]e_2} \text{ (STEPCASER)} \\
\\
\frac{e \mapsto e'}{\text{abort } e \mapsto \text{abort } e'} \text{ (STEPSEARCHABORT)}
\end{array}$$